

How to Find The Right AI Product Metrics

 Generic metrics like “Helpfulness”  Application-specific metrics

Generic AI Metrics Don’t Work

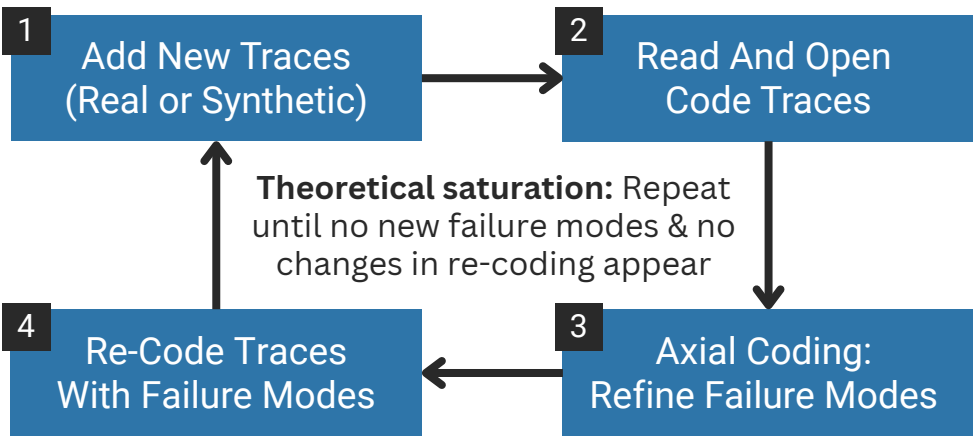
“Generic metrics, such as “hallucination” or “toxicity” often miss domain-specific issues. Their **abuse is endemic** in the industry. The most successful teams **look at data**, identify **failure modes**, and let **app-specific metrics** emerge bottom-up.



Hamel Husain and Shreya Shankar
ML Engineers, 25+ years of experience building & evaluating AI systems.

How to Perform Error Analysis

Error analysis is the highest ROI AI engineering activity. You analyze **LLM traces - full records** of the pipeline execution: user query, input, reasoning, tool calls, and the output.



 As a rule of thumb, you need **~100** high-quality, diverse traces. Those can be real data, synthetic data, or both coded with pass/fail and failure modes.

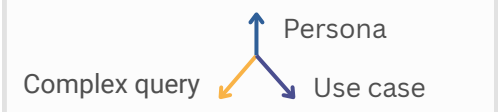
Step 1: (Optional) Generate Synthetic Traces

 Don't generate synthetic data without **hypotheses** about where AI might fail. You can build intuition by using the product. Involve **domain experts**, especially in complex domains.

Synthetic Queries Generation

- ☐ **Prerequisite:** Start by defining at least 3 dimensions that represent where the app is likely to fail
- ☐ **Generate Tuples:** Generate 10-20 random combinations (tuples)
- ☐ **Human Review:** Remove duplicates, unrealistic combos
- ☐ **Generate Queries:** Generate a natural language query for each tuple
- ☐ **Human Review:** Discard awkward or unrealistic queries

Example: Finance Chatbot

Three dimensions:


Three tuples:

↑ Business	↑ Credit line	↑ Low
↑ Person	↑ Mortgage	↑ Low
↑ Person	↑ Credit card	↑ Med.

Two synthetic queries:
Our company is looking for a credit line up to \$250K. We're not your customer. What should we do?
I'd like to take a mortgage. I'm single with high income. I found a house. What's next?

Step 2: Read and Open Code Traces

Open Coding: Write **brief, descriptive notes** with problems, surprises, and incorrect behaviors.

Trace 01:
Bullet points formatting

Trace 02:
Formatting error

Trace 03:
The tool doesn't exist

Trace 4:
I noticed that LLM hallucinates

Step 3: Axial Coding, Refine Failure Modes

Cluster similar notes and let **failure modes** (error categories) naturally emerge. If you use an LLM, always review its output.



Step 4: Re-Code Traces With Failure Modes

Go back and label traces with new failure modes. With more traces, you'll **refine** definitions and **merge** or **split** categories.

...	Request	Response	Result	Failure mode
...	Our company (...)	Here's a plan (...)	Fail	Invalid request categorization ▼
...	Do you reme (...)	I'm sorry (...)	Fail	Request shouldn't be rejected ▼
...	My name is (...)	Information (...)	Fail	Invalid request categorization ▼

Decide What to Measure With Automated Evals

Once you perform initial error analysis, you can implement **automated evaluators** for single failure modes - each **evaluating a single metric**.

01. Start With Analyzing Failure Type

**Specification Failure**

- Your instructions were unclear or incomplete
- Fix the prompt** first. Don't build an evaluator yet

**Generalization Failure**

- LLM fails to apply clear instructions correctly
- These are candidates for **automated evaluators**

02. Consider Two Types of App-Specific Evaluators

**Code-Based Evals**

- Logic you write (e.g., Python)
- Objective, rule-based checks such as XML, SQL, Regex
- Fast, cheap**, deterministic

**LLM-as-Judge Evals**

- Complex or subjective checks
- Uses another LLM as judge
- A single, narrow failure mode
- Start with **binary checks**

It's critical to **align** LLM-as-Judge evals with human experts. Measure:

- TPR** (True Positive Rate) - the most impactful metric to max. alignment
- TNR** (True Negative Rate) - helpful metric